

M9：生命周期、动态内存、析构与 RAII

CSC3002 基础讲义 · 第 5 册 · 前置：对象、指针、引用、构造函数

核心问题：某个名字还能写在代码里时，它指向的对象是否仍然存在，又由谁负责结束对象的使用并释放资源？学完应能画出自动对象与动态对象的不同寿命，正确配对 new/delete，识别悬空访问与泄漏，并解释 RAII 和智能指针的基本用途。

主线：作用域与生命周期 → 自动/静态/动态对象 → new 与释放 → 别名和所有权 → 动态数组与二维表示 → 析构和 RAII → const/static 的位置 → 三种智能指针 → 复制与资源。采用 C++17，以 L7 为范围，配合 L1/L3 与 E22 Q12。错误片段只分析。A1 是固定容量类型，不把它改造成动态字符串。

阅读依据与课程定位

这是对实际课程材料重新核对后的修订版，核对日期为 2026-10-08。L 后的数字是课件代号；“第几张”按文件实际幻灯片顺序计数，从封面开始，不沿用旧模板页脚。E22 页码为 PDF 页码。

- L1: 1-CSC3002_Lecture Notes Overview of C++(5).pptx
- L3: 3-CSC3002_Lecture Notes Functions and Libraries Part 2.pptx
- L7: 7-CSC3002 Dynamic memory management.pptx
- E22: CSC3002 Final 2022 Spring.pdf
- A1: assignment1.pdf

本册学习任务	原材料定位	要能独立完成的动作
作用域与寿命、堆栈图	L1 第 29-30 张；L7 第 4、18-19 张	分开画局部指针、参数副本与动态目标
动态数组、二维数组和释放顺序	L7 第 5-13 张	数清分配次数，选择对应的释放形式
析构与 RAII、资源错误	L7 第 10-17 张	解释泄漏、重复释放与自动收尾
const 与 static 的不同位置	L7 第 22-26 张；L3 第 34 张；E22 Q12	区分存储期、只读访问和共享成员
unique/shared/weak pointer	L7 第 27-30 张	跟踪所有者数量、观察者和目标存活状态

课件决定本册的概念范围；旧卷用于练题型和深度，不预测今年题目或分值。本册的解释、状态表和答案由教学推导得到；未找到官方答案册，因此不称为官方标准答案。保留的最小例子与一般练习属于自编教学补充；涉及原材料时逐项标明“课件例题”“原题代码”或“题意转述”。

当前 L1 第 2 张写的是闭卷，E22 第 1 页写的是可带一张双面 A4 的限制性开卷，两者不能混作今年规则。六册只覆盖 C++ 基础这一段，旧卷的数据结构、算法和部分继承多态内容仍需后续学习。

1. 生命周期是什么？先与作用域分开

生命周期讨论对象从何时开始存在，到何时结束存在；作用域讨论一个名字在代码的什么范围可被查找找到。两者相关，但不是同一问题。

```
int* p = nullptr;
{
    int n = 5;
    p = &n;
}
```

块内 `n` 是自动存储期的整数对象。离开块后，`n` 的生命周期已经结束；外层名字 `p` 仍可访问，但它保存的旧指针不能再正常用来读取那个已经结束寿命的整数。

指针变量还存在，不保证目标对象还存在。 这是本册最重要的区别。语言对生命周期有精确规定，本册首先研究普通局部对象和常规 `new` 创建的对象，见 [对象生命周期规则](#)。

2. 自动局部对象怎样出现和消失？

```
void work() {
    int a = 3;
    {
        int b = 4;
        a += b;
    }
    // 这里 a 仍然存在，b 的名字已经不在作用域内
}
```

进入相应声明时完成对象初始化；内层块结束，`b` 的寿命结束；`work` 结束，`a` 的寿命结束。对有析构函数的类对象，正常离开相应作用域还会调用析构函数。

“自动”指存储期和清理由语言机制管理，不是指值会自动合理初始化。`int n;` 仍可能没有可读取的初值，`int n = 0;` 才明确给出 0。

同一函数调用多次，每次的普通局部对象通常是各次调用自己的对象，正好接回 M2 的局部调用状态。

3. stack / heap 在课程里怎样使用？

常见实现把自动局部状态放在调用栈，把动态分配的存储放在常说的堆。这个模型有助于画调用状态和理解动态分配。

但 C++ 的语义首先规定存储期、生命周期和可访问性，不保证每个局部对象都实际占一块物理栈空间。编译器可能把值放寄存器、消除不必要对象或进行其他合法优化。

课程常用说法	本册更精确关注的事
栈上局部变量	自动存储期，常随块或调用结束
堆上的对象	动态存储期，需要合适的释放管理
静态区对象	静态存储期，寿命覆盖程序运行的大部分阶段

做语义题先判断对象何时存活，不要只凭一张栈堆示意图推断所有物理细节。

4. new 是什么？它不只是“拿到地址”

普通 `new` 表达式取得动态存储并在其中初始化对象，提供指向该对象的指针。

```
int* p = new int(5);
```

左边 `int* p` 声明指针变量；右边 `new int(5)` 创建动态 `int` 对象，初值 5，并提供其指针。这里至少要分出两个角色：局部指针对象 `p`，以及它指向的动态整数对象。

```

局部 p: 保存指针值
      |
      v
动态 int 对象: 值为 5

```

之后 `*p = 8`；改动态整数，`p = ...`；改局部指针。对象关系仍与 M3 一样，新增的只是目标寿命和释放责任。普通分配失败通常抛出 `std::bad_alloc`，不要把它无条件当成返回 `nullptr`；特殊不抛异常形式不是本册主线。规则可核对 `new` 表达式。

5. 为什么局部指针消失时，动态对象不一定消失？

```

int* makeNumber() {
    return new int(7);
}
int* p = makeNumber();

```

函数内部创建的是动态整数，并把其指针值返回。函数调用结束不会自动销毁该动态整数，调用者仍可通过 `p` 访问它；但调用者必须承担后续释放责任，或者立即交给合适的资源管理对象。

与之对照，下面是错误示例：

```

int* badNumber() {
    int n = 7;
    return &n;
}

```

函数返回后，自动局部对象 `n` 的寿命结束，得到的指针不能再当成有效整数目标使用。两段代码都返回 `int*`，但目标的创建方式和寿命不同；返回类型本身不能保证目标有效。

6. delete 做什么？它不会自动改所有指针

```

int* p = new int(5);
delete p;
p = nullptr;

```

对这里与 `new` 正确配对的 `delete`，动态对象的寿命结束并释放相应存储；若对象为类类型，还会执行其析构。下一行把局部 `p` 设为空是独立操作，不是 `delete` 自动完成的。

```

int* p = new int(5);
int* q = p;
delete p;
p = nullptr;

```

`q` 保存的旧指针值没有因此同步变化。它已经不指向可正常访问的原动态整数。**把一个指针置空，不能修复其他别名。**此后不得经 `q` 读写，也不得再用 `q` 重复释放同一对象。

`delete nullptr`; 是允许的空操作; `delete &local`; 则不能用来释放普通局部对象。释放必须符合来源与配对规则, 见 `delete` 表达式。

7. `new[]` 与 `delete[]`: 数组有自己的配对形式

```
int* p = new int[3]{1, 2, 3};
std::cout << p[1];
delete[] p;
```

`new int[3]` 创建三个动态 `int` 元素, 花括号给出初值; `p` 指向首元素; `delete[] p` 与数组分配形式配对。输出 `p[1]` 得到 2。

分配方式	正确的对应释放形式
<code>new T(...)</code>	<code>delete p</code>
<code>new T[n]</code>	<code>delete[] p</code>

不能认为 `delete[]` 的方括号只是装饰, 也不能对 `p + 1` 使用 `delete[]` 释放原数组。释放使用与分配对应的适当起始指针。

`new int[3]`; 不保证三个整数零初始化; `new int[3]{};` 明确把它们零初始化。初始化与分配需要分别读清。

8. 三种常见错误: 悬空、泄漏、重复释放

悬空访问: 目标寿命已经结束, 却仍经旧指针或引用访问它。返回局部对象地址、`delete` 后再读取, 都属于常见来源。

内存泄漏: 动态资源仍占用着, 却失去了合理释放路径。下面是只用于分析的坏例子:

```
int* p = new int(5);
p = new int(8);      // 丢失第一份分配的地址
delete p;            // 只能释放第二份
```

重复释放: 同一份资源被释放两次, 例如两根别名指针各自 `delete`。该行为不是“第二次什么也不做”。

这三类问题都说明只会保存地址还不够, 还需要回答: 谁拥有资源、谁负责释放、什么时候失效? 接下来 RAII 会把这份责任交给对象。

9. `static` 局部变量是什么? 为什么函数返回后还记得值?

```
int next() {
    static int n = 0;
    ++n;
    return n;
}
```

`n` 的名字仍是函数内局部名字, 但其对象具有静态存储期; 普通函数返回不使它消失。这个固定初值示例从 0 开始, 每次调用后保存更新后的值, 所以连续三次返回 1、2、3。

与之对照, 若去掉 `static`, 每次调用重新建立 `n=0`, 随后加为 1, 每次返回 1。不要把“局部名字”直接等同于“仅活一次调用”。

一般静态局部初始化还有首次执行到声明时进行的规则；常量初始化可能更早完成。本例不依赖初始化发生时刻的复杂差别，只需要看清对象不会每次调用都重新建成 0。

10. static 不是只有一种语境

函数内 static 主要影响该对象的存储期；类内 static 数据成员属于类层面的成员，不是每个对象各有一份；命名空间范围的 static 还与名字的连接可见性有关。

本册先掌握第一种，用来解释跨调用保留计数。遇到其他位置，重新按位置读规则，不能把 static 一律翻译成“全局变量”。

这是与 &、*、const 相同的读法原则：符号或关键字需要结合出现的位置解释。

11. const 和生命周期是两个维度

const 限制通过相应类型访问方式修改对象，不负责延长任意目标的寿命。

```
const int* p = nullptr;
{
    int n = 5;
    p = &n;
}
```

p 的目标访问是只读的，但 n 仍在离开块时结束寿命。此后不能因为 p 含 const 就认为读取安全。

属性	回答的问题
类型与 const 限定	允许通过这条访问路径做什么
生命周期	对象现在还存在吗
所有权	谁承担释放或其他资源管理责任

const 引用绑定临时对象在某些语境有特殊延寿规则，但不能推广成“返回任何 const 引用都安全”。本册直接按实际对象来源和寿命判断，不靠这种口号绕过分析。

12. 析构函数是什么？在对象寿命结束时收尾

析构函数是类对象销毁过程中的特殊成员函数，用来完成该类型需要的清理。名字为 ~类型名，不写返回类型，也没有普通参数。

```
struct Trace {
    int id;
    Trace(int n) : id(n) {
        std::cout << "create " << id << '\n';
    }
    ~Trace() {
        std::cout << "destroy " << id << '\n';
    }
};
```

~ 在这里组成析构函数名，不是整数按位取反。它与普通 ~x 表达式属于不同语境。

```
{
    Trace a(1);
    Trace b(2);
}
```

正常离开块时，已完成构造的自动对象按相反构造顺序销毁：先 b，再 a。因此打印 create 1、create 2、destroy 2、destroy 1。析构机制不只出现在显式 delete 上，自动对象离开作用域也会触发。

13. RAII 是什么？先从“早返回忘记清理”理解用途

若函数有多个 return，手工在每条路径写 delete 容易遗漏。**RAII 的核心做法是让资源的取得与管理对象的初始化关联，让资源的释放跟随管理对象的析构。**名称 Resource Acquisition Is Initialization 可以译为“资源取得即初始化”。

需要管理的资源不只有内存，还可能是文件句柄、锁等。类的构造和析构建立清晰责任，自动对象正常离开作用域或发生标准的异常栈展开时，析构机制负责收尾。

```
进入作用域，创建资源管理对象
→对象持有资源并提供访问
→正常返回、提前返回或异常展开
→管理对象析构，释放其拥有的资源
```

这不是“所有对象都放到栈上就没有动态内存”，而是可以用自动寿命的管理对象，管理动态资源。标准容器、字符串和智能指针都是常见的现成工具。

14. unique_ptr：让一个管理对象独占资源

`std::unique_ptr` 表示独占所有权的智能指针：同一份资源由一个这样的所有者负责释放。使用 `<memory>`。

```
std::unique_ptr<int> p = std::make_unique<int>(5);
*p = 8;
```

`std::unique_ptr<int>` 是管理 int 的类型；`make_unique` 创建并初始化动态整数，返回相应所有者；`*p` 提供目标访问。p 离开作用域时自动释放所拥有的整数，无须手写 delete。

```
int readNumber(bool early) {
    auto p = std::make_unique<int>(5);
    if (early) {
        return *p;
    }
    *p = 9;
    return *p;
}
```

auto 在这里由初始化表达式推断 p 的类型为 `unique_ptr<int>`。两条正常返回路径都会让局部 p 析构；返回的是复制出来的 int 值，不是对已释放对象的引用。

15. get、移动与 reset 分别干什么？

```
auto p = std::make_unique<int>(5);
int* observer = p.get();
auto q = std::move(p);
```

get() 提供不转移所有权的普通指针，用于短期观察；不要由 observer 再 delete 目标。第三行通过移动构造把所有权从 p 转给 q，随后 p 为空，q 管理原整数。std::move 本身提供可移动语境，实际转移由 unique_ptr 的移动操作完成；使用时包含 <utility>。

不能写 auto q = p; 复制一个独占所有者，因为这会让两个对象都试图独占同一份资源。q.reset(); 释放 q 当前拥有的资源并令 q 为空，此后旧 observer 不再是可正常访问原整数的指针。

智能指针管理释放责任，不会自动追踪并修复所有借出的裸指针。原始类型与所有者类型的区别可查 unique_ptr。

16. shared_ptr 和 weak_ptr：理解基本用途即可

shared_ptr 允许多个所有者共同管理一份资源，最后一个拥有者释放所有权时销毁所管理对象。它适合确实需要共享所有权的场景，不是所有指针的默认替代品。

```
auto a = std::make_shared<int>(5);
{
    auto b = a;
    *b = 8;
}
std::cout << *a;
```

b 是共享所有者，访问同一个整数。b 离开块并没有使 a 的目标消失，最后输出 8；等最后的所有者 a 也结束时才完成相应资源释放。

weak_ptr 是不保持共享对象存活的观察关系，可用于避免共享所有权环。使用时通过 lock() 尝试取得临时 shared_ptr，并检查是否成功。当前只需分清“独占拥有”“共享拥有”“不拥有的观察”，不展开复杂循环结构实现。

17. 为什么复制一个裸指针资源类可能危险？

假设某类的 int* 成员拥有一个 new 分配，并在析构里 delete。默认复制仅复制地址，于是两个对象都认为自己负责同一个目标。先销毁一个使另一个悬空，最后还可能重复 delete。

问题不是“复制语法有错”，而是“复制后的所有权关系没有定义清楚”。真实设计应选择合适语义：禁止复制、做独立内容复制、或者使用明确共享的管理方式。包含 unique_ptr 的类型默认不能按普通复制取得两个独占所有者，正是在类型层面表达这个约束。

A1 当前 data 是固定字符数组成员，没有额外 new 分配；不要把需要手工 delete 的动态资源模型硬套到它的字符存储上。根据实际成员类型决定责任，才是最小正确设计。

18. 完整程序：作用域结束与所有权转移

```
#include <iostream>
#include <memory>
#include <utility>

struct Trace {
    int id;
    Trace(int n) : id(n) {
        std::cout << "create " << id << '\n';
    }
    ~Trace() {
        std::cout << "destroy " << id << '\n';
    }
};

int main() {
    Trace a(1);
    {
        auto p = std::make_unique<Trace>(2);
        auto q = std::move(p);
        std::cout << (p == nullptr) << ' '
                  << q->id << '\n';
    }
    std::cout << "leave inner" << '\n';
}
```

输出依次为 create 1、create 2、1 2、destroy 2、leave inner、destroy 1。内层 q 销毁时释放 Trace(2)，p 已为空；外层 a 最后销毁。所有权转移没有再创建一个 Trace(2) 副本。

回到 lecture notes：把 L7 的内存图真正推一遍

A. 第 7-8 张 createIndexArray：返回指针不等于返回局部数组

课件任务是创建长度 n、每个元素等于自己下标的数组。下面按课件整理，约定 n 为正整数：

```
int* createIndexArray(int n) {
    int* array = new int[n];
    for (int i = 0; i < n; ++i) {
        array[i] = i;
    }
    return array;
}
```

array 是函数中的局部指针，new 创建的 n 个元素是另一批动态对象。return 复制指针值给调用者；局部指针寿命结束，动态元素仍存在。int* digits=createIndexArray(3); 后得到内容 0、1、2，使用完以 delete[] digits 释放。

若把 3 改成 1，仍然需要 delete[]，因为决定配对的是 **new 的数组形式**，不是元素多于一个。若改成 int array[n];，标准 C++17 的内建局部数组长度不能由这种运行时参数决定；即便某编译器接受扩展，返回指向该自动局部存储的指针也会悬空。

B. 第 9 张的 int**：先有一排指针，再有每一行元素

取两行三列作为最小示意：

```
int** a = new int*[2];
for (int i = 0; i < 2; ++i) {
    a[i] = new int[3]{};
}
```

第一行的 `new int*[2]` 创建两个 `int*` 元素，返回指向其首元素的 `int`。`a[0]`、`a[1]` 各自再保存一行整数的首地址。
类型逐层读：**a 是 int**，`a[i]` 是 `int*`，`a[i][j]` 是 `int`。

```
局部 a --> [行指针0][行指针1]
           |       |
           v       v
         [0 0 0] [0 0 0]
```

每一行内部连续，但两次 `new` 得到的两行之间不保证相邻。这与 `int plain[2][3]` 的整体连续内建数组类型不同，不能把二者都叫“二维”就当成同一种指针类型。

按课件 `DeleteMatrix` 的顺序：先 `delete[] a[0]`、`delete[] a[1]`，再 `delete[] a`。先释放外层指针表会丢掉继续找各行的入口；只释放外层也不会自动递归释放各行。此处为了理解模型约定分配成功，暂不扩展分配中途异常的处理代码。

C. 第 13 张：也可以用一段连续存储表示矩阵

课件另一版 `Matrix` 用 `int* pData` 保存 `rows*cols` 个整数。第 `i` 行第 `j` 列映射到 `pData[i*cols+j]`，下标从零开始。

例如两行三列，先跳过 `i` 个完整行，每行 `cols=3` 个元素，再前进 `j` 个。因此 (1,2) 映射到下标 $1 \times 3 + 2 = 5$ ；(0,2) 映射到 2。这里总共一次数组分配，释放时只需一次 `delete[]`。

课件 `operator()(int i,int j)` 返回 `int&`，所以 `m(1,2)=9`；左边表示实际元素，而非独立整数副本；

`m(1,2)=m(0,1)=100`；先把右侧元素设为 100，再把相同结果写到左侧。运算符调用模型与 M8 完全相通。

这段 `Matrix` 负责在析构中释放 `pData`，但原示例没有完整定义拥有资源时的复制行为。不能据此认为默认复制安全；浅复制会使两个对象指向同一数组并都试图释放。这里把它当构造/析构的教学片段，不把它当完整通用矩阵库。

D. 第 19 张的 nonsense：三种传参在一张图里

课件核心片段：

```
void nonsense(int list[], Point pt, double& total) {
    Point* pptr = new Point;
    list[1] = pt.x;
    total += pt.y;
}
```

调用前 `main` 的 `total=0.0`，`pt.x=1`、`pt.y=2`，`array` 指向 `new int[5]` 创建的数组，然后调用 `nonsense(array,pt,total)`。

`list` 参数调整为 `int*`，指针值是副本，但它仍访问 `main` 的同一动态数组；`Point pt` 按值接收一个独立对象副本；`double& total` 绑定 `main` 的原 `total`。所以函数使 `array[1]` 变为 1，`main` 的 `total` 变为 2.0，`main` 的 `Point` 不因传值自动被改。

局部 pptr 又保存了另一份动态 Point 的位置。函数返回会移除局部指针，却没有 delete 那个 Point，因而泄漏；main 的原代码也没释放 array，另有一份动态数组未清理。new int[5] 未初始化的其他元素不能随意读出“应该为 0”。用堆栈图是为了看清这些不同对象，不能把图中的问号当成一个数值。

E. 第 24-26 张 static：局部保留状态与类共享状态

L3 第 34 张、L7 第 24 张的 initRandomSeed 使用 static bool initialized=false。第一次调用检查为 false，设置种子并改成 true；以后看到同一个状态，不重复初始化。局部名字不变成全局可见名字，但对象跨调用存在。

类的静态数据成员是另一种语境：

```
class A {
public:
    static const int c = 100;
    static int i;
    int k = 0;
    static int f3() { return i; }
};
int A::i = 2;
```

这是从 L7 第 26 张取出的必要部分。A a,b 后，a.k、b.k 是各自的成员；a.i、b.i 则访问同一个共享 i。执行 a.i=3，b.i 也读到 3；并没有给每个实例同步发送一次赋值。

静态成员函数没有当前对象 this，因此不能凭空访问某个对象的 k；但若显式传入 A& obj，仍可以通过 obj 访问有权限的非静态成员。课件“只能访问静态成员”是对不提供对象、直接使用成员名这一场景的简述。

命名空间范围的 static 通常给变量或自由函数内部链接；这与局部 static 的跨调用状态不同。普通命名空间范围 const 对象通常本就具有内部链接，也不能把这些规则一律说成“作用域延长”。

F. 第 28-30 张：智能指针管理谁的寿命？

课件 unique_ptr 数组例子用 auto p=std::make_unique<int[]>(5);，p[i]=i，输出 p[3] 得 3。p 离开作用域时释放其数组，数组形式的所有者选择合适的数组删除行为。使用者不再手工 delete[] p。

shared_ptr 示例先 make_shared<unsigned long>(5)，只有 p 一个所有者，计数为 1。内层 auto p1=p 后两个所有者，计数为 2；p1 离开块后恢复 1，目标仍存在。计数是所有权对象数量，不是整数目标的值，也不是所有裸指针/引用数量。

weak_ptr 不计入强所有者数。课件创建三个 weak_ptr 都不使强计数由 1 变 4；wp.lock() 成功则返回一个临时的 shared_ptr 所有者，持有期间强计数增加。lock 是尝试取得有效共享所有权，不是给整数上互斥锁；不会因此自动让对整数的并发修改安全。

```
auto sp = std::make_shared<int>(1000);
std::weak_ptr<int> wp = sp;
if (auto hold = wp.lock()) {
    std::cout << *hold;
}
```

成功时输出 1000。如果最后一个强所有者已经销毁目标，lock 返回空的 shared_ptr，条件为假，不能再解引用。这样才把“观察者可能晚于目标”这一寿命问题接回基本指针模型。

练习

题 1：名字还在，目标还在吗？

`int* p; { int n=5; p=&n; }` 之后能否正常读取 `*p`？问题是 `p` 的名字不见了，还是目标寿命结束了？

题 2：配对释放

分别为 `new int(5)` 和 `new int[5]{}` 选择释放形式。能否对普通局部变量 `int n;` 写 `delete &n;`？

题 3：别名与置空

`p`、`q` 同指一个 `new` 创建的 `int`。执行 `delete p; p=nullptr;` 后，`q` 是否自动变空？能否再读取 `*q`？

题 4：static 计数

第 9 节 `next` 连续调用三次的结果是什么？去掉 `static` 后呢？

题 5：unique_ptr 转移

`auto p=std::make_unique<int>(6); auto q=std::move(p);` 后谁拥有整数？能否用 `*p` 读 6？`q` 离开作用域时怎样清理？

题 6：const 能延长局部对象寿命吗？

函数返回对局部 `int` 的 `const int&`，调用者是否因为“只读”就可以安全继续读取？

题 7：析构顺序

在一个块中顺序创建自动对象 `Trace a(1)`、`Trace b(2)`、`Trace c(3)`，然后正常离开块。析构输出顺序是什么？

练习详细解析

题 1 解析

不能正常读取。外层 `p` 的名字和指针对象仍存在，但 `n` 的寿命在块结束时结束，`p` 不能再用来访问原整数。诊断的是目标有效性，不是 `p` 的名字查找。输出偶尔看似仍为 5 也不能证明访问合法。

题 2 解析

单对象 `new` 对应 `delete`；数组 `new[]` 对应 `delete[]`。不能 `delete` 普通局部变量地址，因为它不是相应动态分配取得的对象。类型都是 `int*` 也不意味着分配来源相同。

题 3 解析

`q` 不自动变空，仍保存旧指针值，但目标已经被释放，不能读取 `*q`。`p=nullptr` 仅写入指针对象 `p`，不追踪其他别名。也不能再 `delete q`，否则尝试重复释放同一分配。

题 4 解析

`static` 版依次返回 1、2、3；非 `static` 版每次重建 `n=0`，均返回 1。区别是存储期与跨调用保留状态，不是 `++` 运算的规则变化。

题 5 解析

移动构造使 `q` 独占原整数，`p` 变为空，因此不能解引用 `p`。`q` 离开作用域时，其析构释放该整数。`p` 和 `q` 是两个管理对象，但此后只由 `q` 拥有目标。

题 6 解析

不可以。`const` 限制修改能力，不使函数内局部整数在返回后继续存活。返回引用会悬空；若只是需要整数结果，应按值返回。不要把某些临时对象绑定语境的特殊延寿规则套到返回局部对象引用上。

题 7 解析

依次 `destroy 3`、`destroy 2`、`destroy 1`。这里指同一块内按顺序成功构造的普通自动对象，正常离开作用域时反向销毁。若涉及动态所有权或异常中途构造，需要再按相应对象的实际完成构造与释放时刻分析。

做寿命题时的四个问题

对象怎么创建？现在是否仍存活？谁拥有资源并负责释放？当前访问路径是否有效且允许该操作？把这四项分开，就不会把 `const`、指针存在和资源所有权混为一谈。

课件与旧卷练习详细解析

R1. E22 Q12 (第 3 页) : 让局部变量跨调用保留值

原题选项为 const、extern、static、protected，答案 **c: static**。const 管修改能力；extern 主要用于链接声明；protected 管类成员访问，它们都不承担“保留同一个局部状态到下一次调用”的这里的任务。

最小对照：`int n=0; return ++n;` 每次调用返回 1；改为 `static int n=0` 则连续调用返回 1、2、3。判断先看存储期，再推 ++，不要把保留状态误解成函数结果被缓存。

R2. L7 第 8 张: n=1 为什么仍要 delete[]?

因为分配是 `new int[n]` 的数组形式；n 的数值不改变它的形式。对应的数组删除必须写 `delete[]`。把它写成 `delete` 是配对错误，不能用“只有一个元素所以差不多”论证。

R3. L7 第 9、13 张: 分配次数与连续性

两行三列的行指针方案有三次分配：一次两个 `int*` 的表，两次各三 `int` 的行；先释放两行，再释放表。平铺方案只有一次六 `int` 的数组分配，元素 (1,2) 位于下标 5，一次 `delete[]` 清理。

同样六个逻辑元素，不保证物理组织相同。先看实际 `new` 表达式和成员类型，不能只凭“都是矩阵”推断所有权和地址步长。

R4. L7 第 14 张: 一个赋值造成两个问题

课件错误片段的核心是：

```
char* p1 = new char;
char* p2 = new char;
p2 = p1;
delete p1;
p1 = nullptr;
// delete p2; // 错误：再次释放同一对象
```

起初有两份分配。p2=p1 后，原来由 p2 保存地址的第二份分配失去入口，已经泄漏；现在两根指针指向第一份分配。delete p1 释放第一份，p2 成为悬空观察；p1=nullptr 只改 p1，不改 p2。若再 delete p2，就是重复释放。不能只找最后一行而漏掉前面的泄漏。

R5. shared_ptr 与 weak_ptr 的状态题

初始 `sp=make_shared<int>(7)`，`wp=sp`，然后进入块并令 `hold=wp.lock()`，之后离开块，再 `sp.reset()`。分别给出强计数和目标状态。

解析。起初强计数 1，wp 不增加它。lock 成功后 hold 成为第二个所有者，计数 2；离块 hold 销毁后为 1，目标仍存在；reset 让 sp 释放最后一个强所有权，目标销毁。wp 仍可作为观察关系存在，但 expired 为 true，随后 lock 返回空。weak_ptr 的存在没有延长整数寿命。

L7 第 20 张仅要求认识 C 风格动态分配，明确不要求为该局部内容记忆考试细节。本册把精力放在 new/delete、寿命图和 RAII；这不代表其他所有内存题都可以跳过。

A1 的 data 是固定数组成员，不单独拥有 new 出来的缓冲区。能区分这点，就不会为了展示动态内存而破坏题面要求或引入不必要的释放责任。